

Data Model for the Real-Time Cybersecurity Event Analytics Platform

Group 4(B23EE1094, B23ES1032, B23EE1041)

1. Data Model Overview

Aspect	Chosen approach
Data source	Self-generated (Attack Simulator)
Data type	Semi-structured JSON at generation time
Data retrieval	Push-based continuous consumption (Kafka to stream processor) and pull-based SQL for dashboard/forensics
Data storage	Dual persistence - Parquet files(data lake, raw)+PostgreSQL(data warehouse,curated)
Data arrangement	Lake(partitioned by event_type/date/hour) & Warehouse(normalized+aggregates tables)

Data source: Events originate from our own Attack Simulator rather than a third-party feed. It gives us full control over event volume, timing and the mix of normal versus attack traffic, letting us exercise specific detection rules on demand rather than waiting for real incidents to occur.

Data type: Each event is a semi-structured JSON object combining categorical fields (source_type, status), numeric or identifier fields (ports, IDs), free-text fields (usernames, raw payload), and timestamps (event time and ingest time). This heterogeneity is why events are captured as JSON at the point of generation, even though they are later converted to a strict columnar schema for storage.

Data retrieval: Two distinct access patterns exist. The stream processor performs continuous, low-latency, push-based retrieval by consuming from a Kafka topic. The dashboard and any analyst performs on-demand, pull-based SQL retrieval against curated tables.

Data storage: We use dual persistence: raw events are stored as Parquet files (the data lake), while cleaned, aggregated and rule-derived results are stored in PostgreSQL (the data warehouse). This separation keeps the two storage layers optimized for their respective access patterns instead of forcing one engine to serve both.

Data arrangement: In the lake, files are partitioned by event type and by date and hour, so that queries touching a specific day or category can skip irrelevant files entirely. In the warehouse, data is arranged into normalized, fact-style tables, an alerts table and an aggregates table.

2. Architecture

The end-to-end architecture begins with the Attack Simulator, which sends events over HTTP to a Python backend acting as an event API. The backend produces each event onto a Kafka topic, decoupling event generation from processing so that bursts of simulated attack traffic do not block the simulator itself. A Python stream processor consumes this topic continuously and is the only component that writes to persistent storage: it appends raw, unmodified events to the Parquet-based data lake for full historical retention, and it writes curated alerts and windowed aggregates to the PostgreSQL data warehouse. A monitoring dashboard reads from the warehouse for live views and can optionally reach into the lake for deeper forensic queries. Two components are marked optional in the diagram: a DuckDB query layer over the lake, and MinIO as a self-hosted, S3-compatible object store in place of a plain local folder.

Real-Time Cybersecurity Event Analytics — Architecture

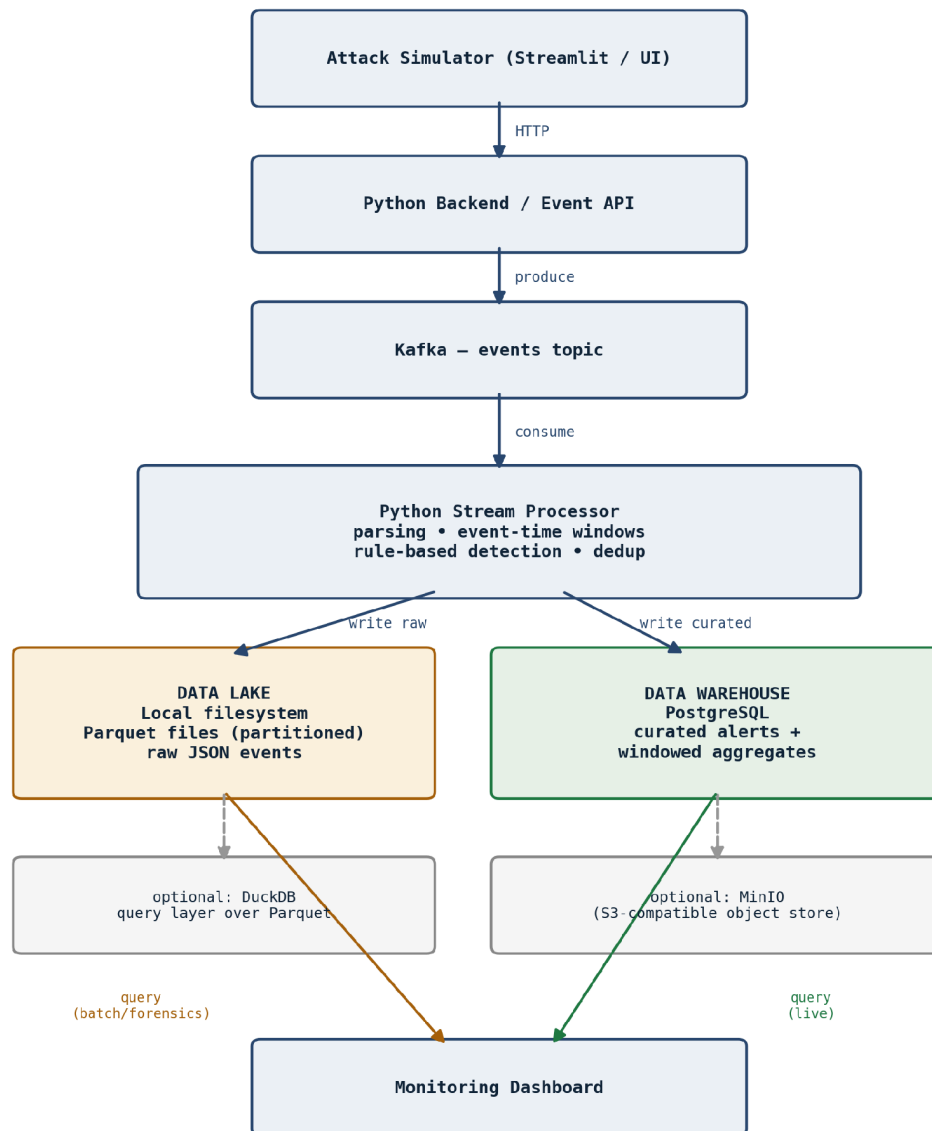


Figure 1. End-to-end architecture: ingestion, dual storage and optional add-ons.

3. End-to-End Data Flow

After the simulator generates an event and the backend produces it to Kafka, the stream processor consumes it under Kafka's at-least-once delivery guarantee, meaning the same event can, in principle, be delivered more than once. Three responsibilities inside the stream processor address the realities of streaming data- First, each event is assigned to a window using its own *event_time* field rather than the time it happened to arrive, since network and queueing delays mean arrival order is not a reliable proxy for what actually happened first. Second, a predefined rule such as "assume no event is more than ten seconds late" determines when a window can be safely closed and its result emitted, and events arriving after this window limit are dropped. Third, a short-lived set of recently seen *event_id*'s is used to discard duplicates before they reach either storage layer. Only after these steps does rule evaluation run, and only its output is written onward. The raw record is appended to data lake (parquet files)

regardless of whether it triggered a rule, while a row is written to the data warehouse (Postgres) only when a rule fires or a window completes. This separation is why the warehouse can stay small and fast while the lake keeps the complete, replayable history.

4. Raw Schema to Curated Schema

The raw event carries an identifier, both timestamps, a source-type category, network and user identifiers, a status, and an unparsed payload field for anything not yet normalized. The lake stores this schema unchanged, one Parquet file per partition, which is why no separate database is required to define it: the schema lives in the files themselves, and any reader (DuckDB, pandas, or a future Spark job) can infer it directly from the Parquet metadata. The warehouse, in contrast, holds two purpose-built tables rather than a copy of the raw schema: an alerts table recording which rule fired, for which entity, over which window, at what severity, and an aggregates table recording per-window counts and unique-entity statistics regardless of whether an alert fired. Both are derived, not raw, which is precisely the justification for keeping them in a different, query-optimized store from the lake.

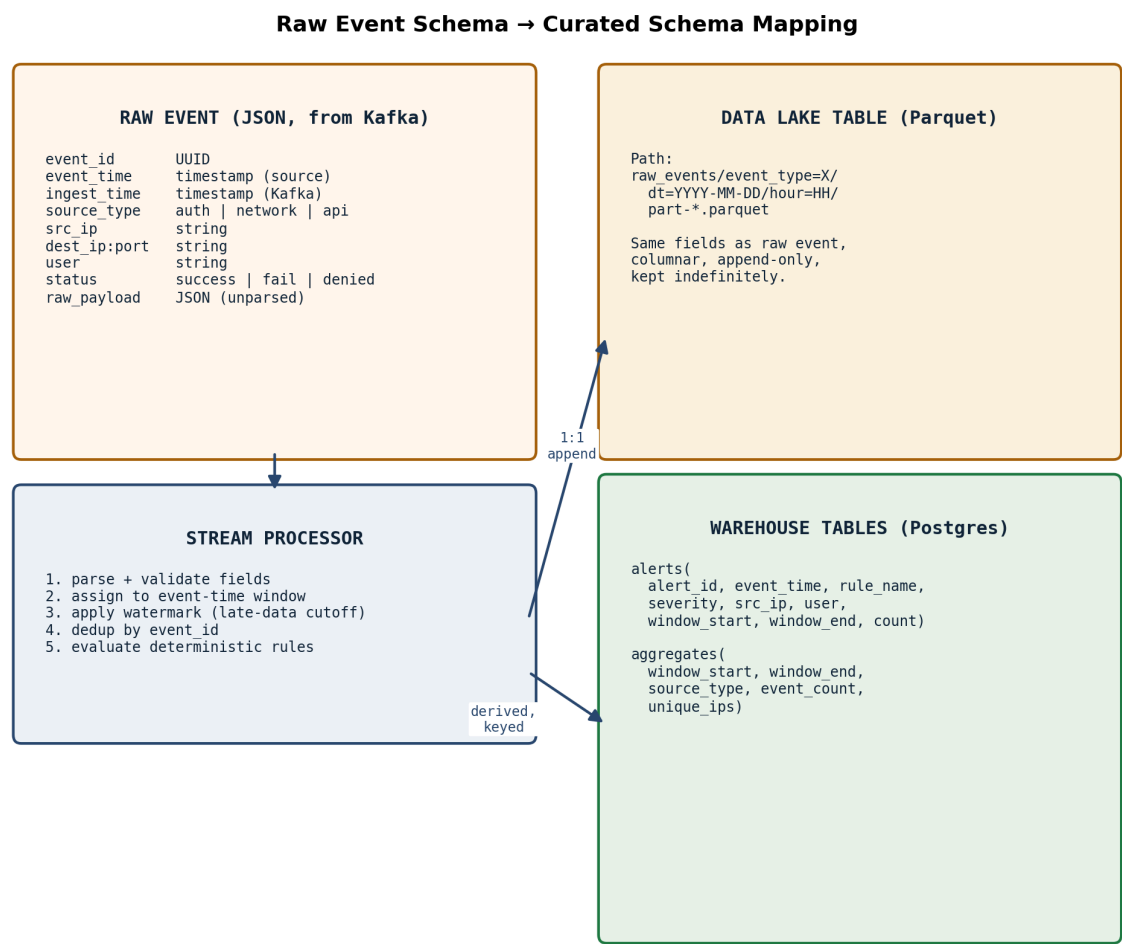


Figure 2. Raw event schema (lake) versus derived, query-optimized schema (warehouse).

5. Justification behind the choices

A MongoDB-based data lake was considered initially, but we chose Parquet because it provides a lightweight, file-based format for retaining raw event data without requiring a database server.

Also, CSV and newline-delimited JSON were also considered as simpler raw-storage formats. We chose Parquet because its columnar format is better suited to analytical workloads: queries can read only the required columns and when data is appropriately partitioned, avoid scanning irrelevant files. This provides better analytical efficiency while requiring no database server.

A single PostgreSQL instance for both raw and curated data was also considered. We rejected this because the raw event stream can grow much faster than the derived alerts and aggregates. Separating the raw data lake from the PostgreSQL analytical layer allows each layer to be optimized for its respective workload.

For the warehouse layer, we chose PostgreSQL because our curated data is structured and relational, consisting primarily of alerts, time-window aggregates and entity-level statistics. PostgreSQL provides SQL-based querying, indexing, aggregation, and relational integrity and therefore provides the functionality we need for dashboard and historical analytical queries without introducing the operational complexity of a larger dedicated analytical warehouse.

Cloud data warehouses such as BigQuery, Snowflake and Redshift were excluded because they introduce cloud-account dependencies and may exceed free-tier limits.

6. Additional Considerations: DuckDB and MinIO

Two optional components extend this design without changing its core data model. DuckDB is an embedded, zero-install analytical database that executes SQL directly against Parquet files; it does not replace the lake, it is simply the query engine we point at it when ad hoc or forensic questions arise, and it is considerably more convenient than writing custom Python for every such query. MinIO is a self-hosted, S3-API-compatible object store that can run in a single Docker container with no internet dependency; adopting it would mean the stream processor writes Parquet objects to a MinIO bucket instead of a local directory, which is closer to how lakes are deployed in production and would ease a future move to real cloud object storage, at the cost of one more service to keep running during development and demos. Given our team size and timeline, we plan to start with local Parquet files and adopt MinIO only if time permits.

7. References

1. Kreps, J., Narkhede, N., Rao, J. Kafka: A Distributed Messaging System for Log Processing. Proc. NetDB, Athens, Greece, 2011. <https://notes.stephenholiday.com/Kafka.pdf>
2. Melnik, S. et al. Dremel: Interactive Analysis of Web-Scale Datasets. Proc. VLDB Endowment 3(1), 2010. <https://doi.org/10.14778/1920841.1920886>
3. Akidau, T. et al. The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing. Proc. VLDB Endowment 8(12), 2015. <https://doi.org/10.14778/2824032.2824076>
4. Raasveldt, M., Mühleisen, H. DuckDB: An Embeddable Analytical Database. Proc. ACM SIGMOD, 2019. <https://doi.org/10.1145/3299869.3320212>